NETFLEX — FINAL MASTER PRODUCT, ARCHITECTURE & ENGINEERING PROMPT

Consolidated specification incorporating confirmed product decisions and hardware-discovery requirements

Version: 1.0

Product: NetFlex

Deployment context: One residential lodge/apartment property in Nigeria, designed to support additional properties later

Known decisions: Hardware not yet confirmed; access is sold by individual, room, or floor; plans use time-based validity; customer access credentials are vouchers/access codes

Primary objective: Build a real, secure, maintainable Wi-Fi billing and network-access platform. This is not a visual prototype.

0. HOW TO USE THIS SPECIFICATION

Treat this document as the single source of truth for product intent and engineering acceptance criteria. Inspect the existing repository before changing it. Do not blindly recreate the project or overwrite working code.

Work in small, reviewable phases. Start with **Phase 0: Discovery and Feasibility**. Do not jump directly into full implementation, and do not treat missing hardware details as permission to invent capabilities. Return a discovery report first, including what is known, what is unknown, what can safely be built now, and what must wait for a physical network test.

This specification describes the intended end state. It does not mean any integration has already been configured, tested, or approved. Never claim that a feature works until there is evidence from tests or the actual provider/hardware.

When requirements conflict, apply this precedence:

1. Security, legal compliance, and financial correctness.
2. Verified capabilities of the selected hardware and providers.
3. Explicit product decisions in this document.
4. Existing repository conventions, if sound.
5. Sensible defaults documented in the assumptions register.

For every requirement, track one status: **Implemented and tested**, **Implemented but not live-verified**, **Simulated in isolated tests only**, **Blocked by external dependency**, or **Not implemented**.

1. ROLE AND MISSION

Act as a principal software architect, senior full-stack engineer, backend engineer, network automation engineer, product designer, security engineer, DevOps engineer, and QA lead.

Build **NetFlex**, a production-grade managed Wi-Fi billing, voucher, subscription, and network-access-control platform for a residential lodge/apartment property in Nigeria. The property owner has a Starlink internet connection and wants to offer paid access to tenants and authorized guests through plans that apply to:

- **INDIVIDUAL:** access for one tenant or a limited number of that tenant’s devices.
- **ROOM:** access for an explicitly configured room and its authorized occupants/devices.
- **FLOOR:** access for an explicitly configured floor and its authorized occupants/devices.

Customer access is voucher/access-code based. A voucher is a credential, not the subscription itself and not proof of payment. Plans have time-based validity. The exact activation policy must be configurable; the recommended initial policy is that validity begins on first successful activation, subject to a configurable redemption deadline. The owner must approve the policy before launch.

NetFlex must connect the full lifecycle:

Discover network → display eligible plans → create order → collect/verify payment where applicable → issue or validate voucher → create entitlement → provision gateway policy → verify actual network state → monitor sessions and expiry → renew, suspend, revoke, or reconcile.

Do not promise 100% reliability. Engineer for measurable reliability, graceful degradation, recovery, and security.

2. KNOWN DECISIONS AND NON-NEGOTIABLE CONSTRAINTS

The following are confirmed:

- Physical network hardware is **not yet confirmed**.
- The system must support **individual, room, and floor** access scopes.
- Plan validity is **time-based**.
- The customer-facing access credential is a **voucher/access code**.
- The deployment context is a Nigerian property.
- The product needs a public marketing website, a captive/tenant portal, and an administrator/operations portal.
- The visual design must be premium, restrained, mobile-first, and **not card-heavy**.

Do not silently change these decisions.

The following remain unknown and must be discovered:

- Starlink service plan, service address, and whether the intended paid hotspot/community Wi-Fi arrangement is permitted.
- Router/gateway brand, model, firmware, management API, and captive portal support.
- Access point and switch brands/models, topology, VLAN support, roaming behavior, coverage, and capacity.
- Whether each room/floor will use a shared SSID, separate SSIDs, or network zones.
- Number of rooms, floors, tenants, concurrent clients, expected peak traffic, and traffic profile.
- Whether a voucher is purchased by the end user, issued by an administrator, or both.
- Whether room/floor purchasers can share access with a defined group and how group members are authorized.
- Whether time validity begins at purchase, first redemption, or first successful network authorization.
- Price list, plan durations, data caps, speed policies, device limits, refunds, and support contacts.
- Production domain, hosting, payment credentials, SMS/OTP provider, email provider, and monitoring destination.

Maintain an assumptions register. No unknown item may be silently promoted to a fact.

3. OPERATING RULES FOR THE CODING AGENT

1. Inspect the repository, current branch/status, package manager, runtime versions, installed dependencies, tests, naming conventions, and existing deployment before editing.
2. Summarize the repository’s actual state. Do not claim to have inspected files that were not available.
3. Preserve working code. Avoid destructive rewrites unless evidence justifies them and the change is documented.

4. Produce an architecture document, data-flow diagrams, phased plan, assumptions register, risk register, and dependency/blocker list.
5. Research current official documentation for the selected framework versions, Paystack, the selected gateway, captive-portal behavior, and relevant security guidance.
6. Do not invent credentials, provider capabilities, API endpoints, router behavior, or successful test results.
7. Never replace real production functionality with a mock while claiming completion. Mocks are allowed only in clearly isolated tests/development environments and must be impossible to enable accidentally in production.
8. Use strict TypeScript, runtime validation, explicit domain types, controlled errors, reproducible migrations, and meaningful automated tests.
9. Run relevant linting, type checks, tests, and builds after meaningful changes. Report actual command output/results and failures.
10. Do not mark a phase complete while required tests fail or critical unknowns remain unresolved.
11. Maintain a living list of unfinished work and launch blockers.
12. Never expose secrets in source code, browser bundles, logs, screenshots, generated documentation, or committed configuration.
13. Do not claim hardware integration is production-ready until tested against the selected physical gateway and representative client devices.
14. Do not claim internet access works merely because an API returns HTTP 200 or a router command succeeds.
15. Do not claim payment success from a browser redirect or client-supplied status.
16. Use UTC internally and explicit local-time rules for display and activation/expiry.
17. Prefer boring, supportable infrastructure over unnecessary microservices. This is initially one property, so use a modular architecture that can scale without premature complexity.
18. When a requirement cannot be implemented because a dependency is missing, build the clean interface and isolated tests, document the blocker, and do not misrepresent the result as live functionality.
19. Never use destructive database resets in production. Never run unreviewed destructive commands.
20. Before finalizing, review security, accessibility, mobile behavior, operational recovery, and the entire purchase-to-network-access lifecycle.

4. PRE-DEVELOPMENT LEGAL AND FEASIBILITY GATES

Before enabling paid access, verify and document:

- The owner’s exact Starlink plan and the applicable terms permit the intended paid community Wi-Fi/hotspot arrangement in Nigeria. Starlink terms have included restrictions on resale except where expressly authorized, including qualifying Priority Plan community Wi-Fi/hotspot arrangements. Do not assume a residential plan permits resale. Check current official terms and obtain written clarification where needed.
- The property has the authority to provide and bill for the service.
- Nigerian requirements relevant to telecommunications, consumer protection, privacy/data protection, taxes, receipts, and payment handling have been assessed by the owner or qualified adviser. Do not claim regulatory approval without evidence.
- Physical gateway, access points, switches, cabling, power, and topology are adequate.
- Captive portal, session control, voucher authentication, rate limits, and expiry are supported by the chosen hardware/firmware or an explicitly selected compatible architecture.
- Expected user and device capacity is known and tested.
- An operator can regain administrative network access if NetFlex, the cloud, or the internet connection fails.
- Production domain, hosting, payment credentials, OTP/SMS provider, support contact, and monitoring destination are configured.
- The operator has an acceptable customer-support, refund, complaint, and outage policy.

Record evidence, owner, date checked, outcome, and any follow-up action for each gate. If legal authorization or a critical hardware capability is unresolved, do not enable live paid access.

5. PHASE 0 — DISCOVERY FIRST (MANDATORY)

Before full implementation, inspect the repository and produce `docs/discovery-report.md`. It must include:

5.1 Repository audit

- Existing app structure, framework versions, package manager, scripts, tests, database, environment configuration, deployment, and known defects.
- What should be preserved, changed, or removed, with reasons.
- Whether npm workspaces/monorepo structure is appropriate. Prefer npm workspaces if a monorepo is appropriate; do not switch package managers without justification.

5.2 Network/hardware discovery checklist

Collect or explicitly mark unavailable:

- Starlink dish/router model and service plan.
- Existing gateway/router make, model, firmware, and configuration access.
- Access point and switch make/model, firmware, PoE needs, VLAN support, and controller requirements.
- Physical topology: Starlink → router/gateway → switch → access points → rooms/floors.
- Network subnets, DHCP, DNS, NAT, VLANs, guest isolation, and current Wi-Fi configuration.
- Floor plan, wall/building materials, dead zones, target coverage, and likely interference.
- Room/floor count and whether users roam between access points.
- Estimated registered tenants, concurrent devices, peak usage, and expected throughput.
- Power backup and how long the network can operate during power outages.
- Current administrative access and a safe recovery method.
- Whether a dedicated gateway can be introduced without breaking existing service.
- Whether management traffic can use a private LAN/VPN rather than public inbound access.

Do not request passwords or secrets in a report. Use placeholders and a secure secret-entry procedure.

5.3 Hardware-neutral feasibility

Create a capability matrix with rows for:

- Captive portal and redirect support.
- Voucher/local-user support.
- External portal integration.
- API or controller automation.
- RADIUS authentication/accounting.
- Per-user/device bandwidth shaping.
- Group/shared bandwidth policy.
- Concurrent-session limits.
- Session disconnect.
- Usage counters.
- Scheduled expiry.

- VLAN/SSID/network-zone support.
- Health/status access.
- Secure remote management.
- Firmware and vendor support lifecycle.

For each row record: required, supported, unsupported, unknown, evidence/source, test needed, and impact.

5.4 Gateway selection

Evaluate at least:

- **MikroTik RouterOS** as a candidate, subject to model/firmware and feature verification.
- **UniFi** as an alternative if compatible equipment already exists or its controller/captive portal capabilities fit the site.
- The existing gateway, if it can safely meet requirements.
- RADIUS as a possible centralized authentication/accounting component where it adds value.

Do not select hardware solely because it is popular. Compare total cost, reliability, manageability, supported APIs, captive portal behavior, expected client capacity, replacement availability in Nigeria, and the owner's ability to operate it. Provide a recommendation only after collecting enough evidence.

5.5 Deliverables and decision gate

Phase 0 must produce:

- `docs/discovery-report.md`
- `docs/architecture/overview.md`
- `docs/architecture/data-flows.md`
- `docs/network/hardware-capability-matrix.md`
- `docs/assumptions-register.md`
- `docs/risk-register.md`
- `docs/decisions/adr-*.md` for significant architecture decisions
- A phased implementation plan with estimates/risk notes
- A list of decisions the owner must make
- A clear distinction between work that can start now and work blocked by hardware/provider decisions

Do not proceed to live network integration until the selected gateway is known and the proof-of-concept has passed.

6. RECOMMENDED BASELINE ARCHITECTURE

Use this baseline unless repository constraints or documented evidence justify a change.

Frontend

- Next.js App Router
- React
- TypeScript strict mode
- Accessible semantic HTML
- Shared UI/design-token package
- Server rendering for public content where useful
- Client-side interaction only when necessary

- Clear route boundaries between marketing, captive portal/tenant, and admin interfaces
- Responsive, progressive-enhancement approach for weak mobile connections

Backend

- NestJS
- Versioned REST API, such as `/api/v1`
- OpenAPI/Swagger documentation
- DTO validation and explicit response serialization
- Domain-oriented modules
- Centralized exception handling
- Authentication, authorization, throttling, audit logging, and correlation IDs

Data

- PostgreSQL as the source of truth
- Prisma ORM
- Versioned and reproducible migrations
- Database constraints and indexes for important invariants
- Transactions for payment and entitlement workflows
- No floating-point arithmetic for money

Background processing

- Redis and BullMQ, if operationally justified
- Separate durable jobs for provisioning, revocation, expiry, reminders, notifications, reconciliation, and health checks
- Exponential backoff with jitter, bounded retries, idempotency, failed-job inspection, and controlled replay
- No critical workflow dependent solely on in-memory timers

Network

- Strongly typed `NetworkAdapter` interface
- Hardware-neutral domain services
- A real adapter only for verified hardware
- MikroTik RouterOS adapter if selected
- RADIUS integration only where justified
- Separate gateway health/provisioning subsystem
- Network-management traffic isolated from public app traffic

Payments

- Paystack as initial Nigerian payment provider, subject to account approval and current provider documentation
- `PaymentProvider` interface to allow future providers
- Server-side transaction verification
- Raw-body webhook signature verification
- Idempotent event handling
- Reconciliation and controlled refund workflow

Infrastructure

- npm workspaces if monorepo is appropriate
- Managed PostgreSQL with automated backups and point-in-time recovery where available
- Managed Redis or properly secured equivalent
- Independently deployable API and worker processes
- TLS for all public endpoints
- CI for lint, type-check, tests, migration checks, and builds
- Structured logs, error tracking, metrics, and alerting

Suggested layout, adapt to the repository rather than recreating blindly:

```
apps/
web/
api/
worker/
packages/
database/
shared-types/
validation/
config/
ui/
infra/
deployment/
monitoring/
docs/
architecture/
api/
network/
security/
runbooks/
```

Do not split into microservices without a documented reason. A modular monolith plus worker process is a reasonable initial baseline.

7. CORE DOMAIN MODEL AND PROPERTY HIERARCHY

Support this hierarchy:

Property → Building → Floor → Room → Tenant assignment → Devices and network sessions

A property can have one or more gateways, access points, and network zones. A gateway belongs to a property and may serve one or more configured zones. Rooms belong to floors; floors belong to buildings. Tenant-to-room assignments need effective dates and history.

Use explicit access scopes:

- `INDIVIDUAL`
- `ROOM`
- `FLOOR`

Enforce scope in backend services and database queries. A floor entitlement must not automatically grant property-wide access. A room entitlement must not automatically grant access to adjacent rooms. Group membership must be explicit and auditable.

Define who may purchase, own, administer, renew, transfer, cancel, or share each group subscription. Avoid ambiguous “one voucher for the whole floor” behavior. A floor purchaser must be authorized by property policy, and group membership must be defined rather than inferred from knowing a code.

Support:

- Tenant moving rooms/floors.
- Tenant departure.
- Group-member addition/removal.
- Subscription ownership transfer.
- Room/floor reassignment.
- Overlapping subscriptions and precedence.
- Group subscription expiry affecting all included devices consistently.
- Audit history for all these changes.

8. VOUCHER-FIRST ACCESS MODEL

Voucher/access code is the customer’s network credential. Do not confuse it with payment, order, subscription, entitlement, or active session.

8.1 Voucher use cases

Support:

1. A customer pays for a plan and receives a voucher.
2. An authorized administrator generates or issues a voucher according to configured policy.
3. A customer enters an existing valid voucher in the captive portal.
4. Where configured, a voucher can be associated with an individual, room, or floor scope.
5. A group voucher can be used only by the group and devices explicitly authorized under its policy.

Do not allow an administrator-generated voucher to imply that payment occurred. Record whether a voucher is paid, complimentary, manually issued, promotional, or otherwise authorized, including the actor and reason.

8.2 Voucher generation and storage

- Generate codes using a cryptographically secure random generator.
- Use sufficient entropy to resist guessing; choose length and alphabet after usability and entropy analysis.
- Avoid confusing characters if codes are typed manually.
- Do not use sequential or predictable voucher IDs as credentials.
- Store a one-way hash or keyed verifier for redeemable codes rather than plaintext where operationally possible.
- If the system must display a code once at issuance, design a secure one-time display/retrieval flow. Do not log codes or expose them in analytics.
- Apply rate limits and progressive abuse controls to redemption attempts.
- Never expose whether a guessed code is valid through overly detailed errors.
- Allow safe revocation, suspension, expiry, and controlled reissue.
- Do not store voucher codes in URLs, analytics events, or referrer-leaking links.

8.3 Voucher states

Use explicit validated states appropriate to the lifecycle, for example:

- `GENERATED`
- `ISSUED`
- `ACTIVE`
- `REDEEMED` (if one-time redemption semantics are used)

- `EXPIRED`
- `SUSPENDED`
- `REVOKED`
- `EXHAUSTED` (for exhausted device/session allowance)

Document the state machine. Avoid contradictory states. If vouchers can be reused across multiple devices, distinguish “voucher activated” from “voucher redeemed by a device” and model allowed uses explicitly.

8.4 Redemption rules

- Redeem atomically to prevent concurrent requests from overusing a voucher.
- Validate code, status, property, scope, target group, activation deadline, time validity, device allowance, concurrent sessions, payment/issue authorization, and gateway eligibility.
- A paid voucher must not become usable until the associated payment is verified server-side.
- A free/manual voucher must require an authorized issuance record and audit trail.
- Bind a device only according to an explicit policy. Do not assume a MAC address is a durable or privacy-neutral identity; mobile devices may randomize MAC addresses.
- Define how a customer recovers access after changing device, forgetting a code, or reaching a device limit.
- Never let a voucher code alone reveal another tenant’s personal data, transactions, or group membership.
- Group codes do not grant unrestricted membership: use explicit scope, authorized group, device limit, and abuse controls.

8.5 Voucher, order, subscription, entitlement, and session are separate

Model these as separate records/aggregates:

- **Order:** customer’s purchase intent and immutable pricing snapshot.
- **Payment:** provider transaction and financial state.
- **Voucher:** access credential and redemption policy.
- **Subscription:** purchased time-based service contract and plan version.
- **Entitlement:** authorization to access a particular network scope under a policy.
- **Network session:** an observed or gateway-accounted client session.

A voucher may reference a subscription/entitlement but must not become the source of truth for payment status or actual network connectivity.

9. TIME-BASED PLANS AND EXPIRY POLICY

Time-based validity is confirmed. Do not silently introduce a data-only or usage-only plan as the primary model. Optional data caps may be supported only if the gateway provides reliable counters and enforcement.

Plan configuration must include:

- Name and description.
- Scope: individual, room, or floor.
- Price and explicit currency.
- Duration value and unit.
- Activation policy.
- Maximum delay before first activation/redemption.
- Maximum devices and concurrent sessions.
- Upload/download rate policy.

- Optional data cap only where supported.
- Group-sharing policy.
- Optional service hours.
- Auto-renewal availability.
- Publication/availability state.
- Refund/cancellation policy.
- Property/network-zone restrictions.
- Any fair-use terms shown before purchase.

Support duration units explicitly, such as minutes, hours, days, or calendar months where required. Do not implement “one month” as a fixed number of hours if the business intends a calendar month. Define daylight-saving/time-zone behavior where relevant, although UTC remains the stored basis.

Recommended initial rule, pending owner approval:

- Voucher/entitlement validity starts on the first successful network authorization, not merely when the customer opens checkout.
- A configurable activation deadline prevents vouchers from remaining unused indefinitely.
- The expiry timestamp is computed once and persisted, then reused consistently for all devices in the same group entitlement.
- If activation starts at purchase for a particular plan, clearly disclose this before checkout and persist the selected rule.

Use UTC internally. Use the property’s configured time zone for display. Use database/trusted time for expiry decisions. Define whether expiry is inclusive/exclusive and test boundary conditions. Gateway-side expiry should be used as a second enforcement layer where supported, but the application must also reconcile expiry.

Renewals must follow an explicit rule: stack after current expiry, extend from current time, or replace current plan. Do not unexpectedly shorten, reset, or stack validity. Plan version changes must not rewrite existing paid terms.

10. PLAN CATALOG AND PRICING

Administrators can create, edit, version, publish, pause, and archive plans.

Use integer minor units for money and explicit currency codes, with NGN as initial business currency unless configuration requires otherwise. Never use floating-point arithmetic for money. Use server-calculated amounts; ignore browser-submitted prices.

Only published, available plans may be purchased. Validate price, scope, duration, device count, bandwidth, group policy, and property restrictions on the server.

Each plan version used by a paid order must be immutable. Store the relevant plan version, price, duration, scope, policies, and terms snapshot on the order/subscription. New plan edits must not silently alter an existing purchase.

Display the final payable amount, duration, activation rule, device limit, sharing rules, speed policy, and refund/cancellation conditions before payment. Do not promise guaranteed speeds unless the operator has a real enforceable and supportable guarantee.

11. NETWORK HARDWARE DISCOVERY AND ADAPTER DESIGN

Hardware remains unknown. Therefore:

- Build the domain model, interface, test doubles, and configuration validation first.
- Do not hard-code assumed RouterOS behavior into generic business logic.

- Do not expose a fake “connected” state in production.
- Keep any simulator behind an explicit development/test-only configuration guard.
- Do not attempt live network changes before the hardware capability matrix and proof-of-concept are approved.

11.1 NetworkAdapter contract

Design a strongly typed interface with operations equivalent to:

- `healthCheck`
- `validateConfiguration` / `discoverConfiguredGateway`
- `createOrUpdateIdentity`
- `applyPolicy`
- `activateEntitlement`
- `suspendEntitlement`
- `revokeEntitlement`
- `disconnectSession`
- `listActiveSessions`
- `readUsageCounters` where supported
- `readGatewayHealth`
- `reconcileDesiredAndActualState`
- `getCapabilities`
- structured operation results with stable error codes and safe diagnostics

The contract must express unsupported features explicitly. A method must not silently pretend to enforce a policy the gateway cannot enforce.

11.2 Gateway security

- Store gateway credentials in a secret manager or protected environment configuration.
- Use least-privilege accounts.
- Restrict management access with firewall rules and private network/VPN where practical.
- Do not expose management interfaces directly to the public internet.
- Do not assume Starlink provides a stable public inbound IP.
- Never return router passwords, management addresses, unrestricted device details, or raw command output to tenant-facing APIs.
- Never accept an arbitrary gateway URL, internal IP, shell command, or router command from the browser.
- Use timeouts, per-gateway locking where needed, bounded concurrency, idempotent operations, and post-mutation verification.
- Store each network operation with operation ID, gateway, target entitlement, type, desired state, attempts, safe error details, timestamps, and result.
- Do not blindly repeat non-idempotent router mutations.
- After an important change, read back actual policy/session state. An API success response alone is not sufficient.

11.3 Candidate hardware

MikroTik RouterOS is a candidate, not a confirmed dependency. If selected, consult current official RouterOS/HotSpot/API/RADIUS documentation and test the exact model and firmware. UniFi may be suitable where existing compatible equipment and controller capabilities meet the requirements. Compare actual support before selecting.

Use RADIUS only if the chosen architecture benefits from centralized authentication/accounting and the selected gateway supports the required behavior. Document authentication methods, shared-secret handling, accounting, disconnect behavior, and failure policy.

11.4 Proof-of-concept before production

On the selected physical gateway, prove:

1. Captive portal redirect on supported devices.
2. Voucher validation and invalid-code handling.
3. Paid voucher activation only after verified payment.
4. Individual entitlement provisioning.
5. Room/floor scope enforcement.
6. Device and concurrent-session limits.
7. Rate-limit mapping and direction.
8. Session disconnect.
9. Expiry and revocation.
10. Router reboot, application outage, and reconnect behavior.
11. Gateway loss/recovery.
12. Actual browsing from representative Android, iOS, and laptop clients.
13. Reconciliation after interrupted provisioning.
14. Administrative recovery if cloud services fail.

Record hardware model, firmware, configuration, test devices, steps, expected/actual results, and limitations.

12. CAPTIVE PORTAL AND TENANT ACCESS FLOW

The portal must work on mobile devices and low-bandwidth connections.

Expected flow:

1. User connects to the property Wi-Fi.
2. Gateway redirects the device to the captive portal using the gateway's supported redirect mechanism.
3. Portal shows the property identity and a clear connection/purchase action.
4. User chooses an eligible plan or enters an existing voucher.
5. For a purchase, the server creates an order from a validated plan version and target scope.
6. The user completes checkout through the payment provider.
7. NetFlex verifies payment independently.
8. NetFlex issues or enables the appropriate voucher according to policy.
9. User submits the voucher or follows the approved activation flow.
10. The backend validates scope, validity, device/session rules, and payment/issue authorization.
11. A durable job provisions the gateway.
12. NetFlex verifies the actual gateway state.
13. Only then display **Connected**.
14. The user can see validity/expiry and get recovery help.

Required states:

- Welcome
- Choose plan
- Enter voucher
- Phone entry/verification if configured
- Review purchase
- Payment pending
- Payment confirmed
- Voucher issued
- Provisioning in progress
- Connected
- Invalid voucher
- Voucher already used/exhausted
- Expired
- Suspended
- Payment failed
- Gateway unavailable
- Provisioning failed/retry pending
- Support/recovery

Support gateway-specific captive portal parameters and a fallback to open the portal in a normal browser. Do not assume all mobile operating systems open the full browser automatically. Handle captive-portal connectivity checks, redirect restrictions, certificate requirements, and local network access according to selected hardware. Do not promise arbitrary HTTPS traffic can be redirected. Never intercept private traffic, weaken TLS, or bypass device security.

Never claim a connection is active until the gateway confirms that the expected policy has been applied and the client can actually access the permitted network.

13. PAYMENT PROCESSING — FINANCIAL CORRECTNESS IS CRITICAL

Use Paystack initially, behind a `PaymentProvider` interface.

13.1 Order and payment flow

1. Validate plan, plan version, property, scope, target group, activation rule, and customer eligibility.
2. Calculate amount server-side.
3. Create a durable order before checkout.
4. Create a unique internal payment record and provider reference.
5. Initiate provider checkout and return only necessary client-safe details.
6. Treat browser return/redirect as a UX event only.
7. Verify webhook signature using the provider’s current documented method and original raw request body.
8. Persist valid webhook events and provider identifiers.
9. Acknowledge valid receipt promptly; process business effects asynchronously.
10. Verify transaction status and match amount, currency, reference, recipient/account, and order ownership against provider-side data.

11. Process the verified event idempotently.
12. Atomically record the financial success and create the intended subscription/voucher/entitlement records.
13. Queue network provisioning.
14. Expose accurate progress.
15. Reconcile local records with provider state periodically.

13.2 Payment state model

Use explicit states such as:

`CREATED`, `PENDING`, `SUCCEEDED`, `FAILED`, `ABANDONED`, `EXPIRED`, `REFUNDED`, `PARTIALLY_REFUNDED` where supported.

Define valid transitions. Do not allow arbitrary status changes from client input. A late or out-of-order event must be handled according to documented transition rules.

13.3 Webhook and idempotency requirements

- Use the provider's documented signature verification with the unmodified raw body.
- Protect against replay and duplicate delivery.
- Enforce unique provider references and event identifiers where appropriate.
- Do not create duplicate vouchers, subscriptions, entitlements, or network identities when callbacks repeat.
- Acknowledge after durable receipt, not after a long network provisioning operation.
- Use durable event records/outbox or an equivalent pattern so a crash between database commit and queue enqueue cannot lose critical work.
- Use database transactions and unique constraints to protect concurrent callbacks.
- Do not trust amount/currency/reference/status from the browser.
- Reject mismatched amount, currency, recipient, reference, transaction ownership, or unsupported transaction state.
- Never log secret keys or full sensitive payloads.
- Reconciliation must detect discrepancies without silently rewriting financial history.

13.4 Provisioning failure after payment

A paid order with failed network provisioning remains paid. Do not automatically charge again. Keep the order financially successful and the access state pending/failed with a clear recovery path. Retry safe provisioning operations, alert an operator, and offer a documented refund/manual resolution policy. Do not say "connected" until network verification succeeds.

13.5 Refunds

Implement authorized refund requests and provider-side status verification. Record who requested/approved the refund, reason, amount, provider reference, timestamps, and result. Do not automatically revoke valid access based only on an unverified refund notification. Define partial-refund consequences and whether an already-consumed time entitlement can be refunded. Financial records must remain auditable.

14. AUTHENTICATION, VOUCHER IDENTITY, AND SESSIONS

Voucher access is the primary network credential. A customer account is still useful for receipts, renewal, recovery, and group management, but do not make account creation unnecessarily obstructive for guest checkout unless business policy requires it.

Implement:

- Optional/required tenant registration based on approved policy.
- Phone verification through a configured OTP provider where required.
- Secure sign-in and session management.
- Account recovery.
- Session revocation and secure logout.
- Device/session management.
- Administrator MFA.
- Rate limits on OTP requests and attempts.
- Anti-enumeration responses for arbitrary phone numbers.
- Secure, HttpOnly, SameSite cookies with CSRF protection as applicable.
- Session fixation and privilege-escalation protections.
- OTP storage as protected verifiers, short expiry, one-time use, attempt limits, and no plaintext logs.

Do not invent or hard-code an SMS provider. Make the provider configurable and document setup. If OTP credentials are unavailable, keep the integration blocked or isolated in test mode; do not silently bypass required verification in production.

15. ROLES AND SERVER-ENFORCED PERMISSIONS

Roles:

- Platform owner/super administrator
- Property administrator
- Operations/network administrator
- Finance staff
- Support staff
- Tenant
- Room/floor group purchaser or manager, if enabled

Permissions:

- Owner: all authorized property settings and platform configuration.
- Property administrator: assigned property/buildings/floors/rooms/plans/tenants/groups.
- Network administrator: gateways, sessions, provisioning jobs, health, and explicitly authorized network operations.
- Finance staff: transactions, reconciliations, refunds, and permitted reports; no router credentials or unrestricted network controls.
- Support staff: permitted tenant/subscription/payment status and support records; no unrestricted financial or network access.
- Tenant: only own account, vouchers, transactions, subscriptions, devices, sessions, and authorized group data.
- Group manager: only the assigned room/floor group and authorized members; no unrelated tenant information.

Enforce permissions in service methods and database queries. Hiding a button is not authorization. Test cross-tenant and cross-property access. Use least privilege, deny by default, and audit privileged actions.

16. TENANT PORTAL

Provide a functional tenant portal with:

- Account/guest access overview

- Voucher entry, status, and safe recovery
- Current subscription and entitlement status
- Remaining validity and expiry time
- Available plans
- Purchase and renewal
- Transaction history and receipts
- Connected devices/sessions where supported
- Device removal where policy permits
- Room/floor group membership where authorized
- Usage statistics only when real counters are available
- Connection status based on actual gateway data
- Support/help and outage reporting
- Notification preferences
- Privacy controls and secure sign-out

Use truthful, understandable statuses:

- Payment pending
- Payment confirmed
- Voucher available
- Access being configured
- Connected
- Subscription expiring soon
- Subscription expired
- Access suspended
- Gateway temporarily unavailable
- Provisioning needs attention

Never fabricate usage, signal strength, bandwidth, device online status, or estimated expiry. If a metric is unsupported, say it is unavailable. Provide useful recovery instructions without exposing internal infrastructure details.

17. ADMIN AND OPERATIONS PORTAL

Every section must be backed by real API behavior or clearly marked as not yet available. Do not build non-functional buttons or fake dashboard data in production.

Overview

- Gross collections
- Successful transactions
- Refunds
- Net collections only when accounting rules support it
- Active subscriptions/entitlements
- Expiring subscriptions
- Active network sessions where supported
- Gateway health

- Failed provisioning jobs
- Payment reconciliation discrepancies
- Queue backlog and critical alerts

Property management

- Properties/buildings/floors/rooms
- Tenant assignments and history
- Gateways/access points/network zones
- Group memberships and purchaser authority
- Import/export only with validation, permissions, and audit trails if added

Plans and vouchers

- Create/edit/version/publish/archive plans
- Scope, price, duration, activation rule, device/session limits, speed policy, and group rules
- Generate/issue vouchers with permissions and audit records
- View voucher status without unnecessarily revealing active credentials
- Bulk generation only with explicit confirmation, safe one-time export, and limits
- Revoke/suspend/reissue vouchers
- Report unused, redeemed, expired, and revoked vouchers

Subscriptions and entitlements

- Search/filter
- Inspect lifecycle, plan snapshot, payment, voucher, and provisioning history
- Suspend/reactivate
- Revoke with reason
- Extend validity with reason
- Manage permitted devices/group members
- Avoid irreversible bulk operations without confirmation

Payments and finance

- Transactions and provider references
- Pending/failed payments
- Refund requests and outcomes
- Reconciliation discrepancies
- Downloadable reports with access control and export audit
- Clear distinction between gross collections, refunds, and net figures

Network operations

- Gateway health/capabilities
- Active sessions and real usage counters
- Provisioning queue and failed jobs
- Safe retry/replay controls
- Desired-versus-actual policy drift

- Authorized disconnect/suspend/revoke controls
- Emergency actions with confirmation and audit
- No arbitrary router command execution

People and security

- Staff accounts, roles, permissions, MFA, active sessions
- Audit logs
- Failed login and abuse signals
- Support tickets and payment/connectivity complaints

Settings

- Property timezone
- Currency
- Payment and notification provider configuration status
- Expiry/activation policy
- Support contact
- Branding
- Maintenance mode
- Retention policy and privacy notices

All mutations require server authorization, validation, audit logging, and appropriate confirmation.

18. MARKETING WEBSITE AND PRODUCT COPY

Build public pages:

- Home
- How it works
- Plans/pricing
- Property and coverage information
- FAQ
- Contact/support
- Privacy policy
- Terms of service
- Refund/cancellation policy

Keep the call to action direct: connect to the property Wi-Fi, choose a plan or enter a voucher, and get online.

Copy must be accurate. Do not claim:

- Uninterrupted internet
- Guaranteed Starlink speed
- Universal coverage throughout every room/floor
- Perfect security or anonymity
- Real-time usage metrics that are not supported
- Regulatory approval or Starlink authorization that has not been documented

Use SEO-friendly metadata, canonical URLs, social metadata, accessible headings, optimized images, and fast pages. Do not expose private tenant or operational information in public metadata, search results, source maps, or error messages.

Terms and privacy notices must accurately describe what data is collected, why it is collected, how long it is retained, support contact, third-party providers, and applicable user rights. Have the owner obtain appropriate legal review before production.

19. DESIGN SYSTEM — STRICT NO-CARD RULE

The interface must feel premium, calm, modern, deliberate, and consistent. Avoid generic dashboard templates.

Do not use a card-heavy interface. Avoid repeated bordered boxes, floating metric cards, excessive shadows, oversized rounded containers, unnecessary gradients, decorative animations, and arbitrary visual effects.

Prefer:

- Strong typography and clear type hierarchy
- Generous but efficient whitespace
- Full-width section layouts
- Editorial-style marketing sections
- Structured columns and responsive grids
- Fine dividers and restrained backgrounds
- Clean tables and compact forms
- Inline status indicators
- Thoughtful empty/loading/error/success states
- Consistent iconography
- Restrained accent color
- Consistent spacing, radii, borders, and focus states

Create shared design tokens for colors, typography, spacing, radii, borders, shadows (use sparingly), and focus states. Use a coherent visual language across public, captive, tenant, and admin interfaces. Do not use cards as a substitute for hierarchy.

For data-heavy admin pages, use well-spaced tables, section headings, filters, side navigation, inline summaries, and responsive list/table transformations. Use a card only where a distinct grouped object genuinely benefits from one.

Do not invent brand colors or logo assets if the repository already defines them. Inspect existing assets first; document any new brand decision.

20. RESPONSIVENESS AND ACCESSIBILITY

Design mobile-first. Test widths around 320, 360, 390, 430, 768, 1024, and 1440 CSS pixels.

Requirements:

- No accidental horizontal page overflow.
- No clipped primary actions.
- No unusable desktop-only tables on mobile.
- Forms that work with mobile keyboards.
- OTP inputs support paste/autofill where appropriate.
- Clear labels, instructions, and error messages.

- Touch targets around 44 × 44 CSS pixels where practical.
- Keyboard navigation and visible focus.
- Semantic landmarks and heading order.
- Accessible dialogs, menus, and tables.
- Reduced-motion support.
- Sufficient contrast and screen-reader labels.
- Responsive navigation.
- Loading, error, empty, pending, and success states.
- No important action available only on hover.
- Avoid large assets or unnecessary scripts that burden weak mobile connections.

Test real mobile browsers and captive-portal environments. A desktop responsive preview alone is not sufficient. Include Android Chrome and iOS Safari where devices are available, and document any unavailable test environments. Aim for WCAG 2.2 AA where practicable and document exceptions.

21. DATA MODEL AND DATABASE INTEGRITY

Design a normalized, documented schema with foreign keys, unique constraints, indexes, deletion policies, and migration history.

Core entities should include, where appropriate:

- User
- Role
- Permission
- UserRole
- Session
- OTPChallenge
- Property
- Building
- Floor
- Room
- TenantAssignment
- Device
- Gateway
- AccessPoint
- NetworkZone
- Plan
- PlanVersion
- Order
- Payment
- PaymentEvent
- Refund
- Voucher
- VoucherRedemption

- Subscription
- Entitlement
- GroupMembership
- NetworkSession
- ProvisioningJob / NetworkOperation
- UsageSample
- Notification
- SupportTicket
- AuditLog
- IdempotencyRecord
- OutboxEvent or equivalent durable event record
- SystemSetting

Add or change entities based on the actual domain model. Avoid forcing unrelated lifecycles into one table.

Integrity rules:

- Money stored as integer minor units with explicit currency.
- Provider transaction references unique where appropriate.
- Webhook event processing idempotent.
- Voucher verifier unique and never logged.
- Voucher redemption concurrency-safe.
- Tenant ownership enforced in queries and service methods.
- Group scope validated against the property hierarchy.
- Paid subscriptions tied to verified payments and immutable plan snapshots.
- Active entitlements have coherent start and expiry times.
- Plan versions used by paid purchases are immutable.
- Audit records cannot be silently altered by ordinary administrators.
- Foreign keys and deletion behavior preserve financial history.
- Migrations are reproducible and safe to deploy.
- Sensitive data is minimized and retained only as required.
- Use database transactions for multi-record financial/entitlement changes.
- Add constraints where PostgreSQL can enforce invariants instead of relying solely on application checks.

Document indexes and expected query patterns. Use soft deletion only where it serves a documented retention/audit need; do not make every table soft-deletable by default.

22. API DESIGN

Create versioned, documented endpoints with consistent response envelopes, stable error codes, validation, pagination, filtering, sorting, and explicit authorization.

Required areas:

Authentication and sessions

- Registration/onboarding where enabled

- OTP request/verification
- Sign-in/sign-out
- Session listing/revocation
- Account recovery

Public catalog and captive portal

- List published plans
- Read a plan
- Preview eligible purchase options
- Validate voucher without leaking sensitive details
- Redeem/activate voucher
- Read current activation/provisioning status
- Recover from pending payment or provisioning

Orders and payments

- Create order
- Initiate checkout
- Read order status
- Provider webhook receiver
- Verify transaction
- Request authorized refund
- Reconcile payments (restricted)

Vouchers

- Issue/generate vouchers (restricted)
- View permitted voucher status
- Redeem voucher
- Suspend/revoke/reissue (restricted)
- Review redemption history (restricted)

Subscriptions and entitlements

- List current user’s subscriptions
- Read status and entitlement history
- Purchase/renew
- Manage permitted devices
- Request cancellation where applicable
- Group membership management within permissions

Property and groups

- Read authorized building/floor/room details
- Manage tenant assignments and groups within permissions
- Validate target scope

Network

- Gateway health/capability status (restricted)
- Provisioning status
- Authorized network operations
- Session inspection
- Usage retrieval where supported
- Desired-versus-actual reconciliation

Administration and support

- Manage users/plans/properties/groups
- Inspect payments and subscriptions
- Review audit logs
- Manage support tickets
- View operational health

Internal jobs and webhooks must not be accessible through tenant permissions. Never expose unrestricted network details to public endpoints. Document request/response examples and authorization rules in OpenAPI.

23. BACKGROUND JOBS, OUTBOX, AND RECONCILIATION

Implement durable, observable jobs for:

- Network provisioning
- Network revocation
- Subscription/voucher expiry
- Expiry reminders
- Payment reconciliation
- Webhook event processing
- Network-state reconciliation
- Notifications
- Usage aggregation where supported
- Gateway health checks
- Safe cleanup of eligible temporary records

Use idempotency keys and unique job identifiers. Ensure database state changes and job scheduling cannot silently diverge; use an outbox pattern or another durable mechanism where appropriate.

Retries must be bounded and observable. Manual replay must require authorization and must not duplicate payments, subscriptions, refunds, vouchers, or network identities.

Periodically reconcile:

- Provider payment state versus local payment records
- Payments versus orders/subscriptions/vouchers
- Active entitlements versus actual gateway policy
- Expected versus observed sessions
- Subscription expiry versus actual access

- Desired gateway configuration versus actual configuration
- Voucher state versus allowed redemptions

Report discrepancies. Do not silently overwrite financial history or blindly revoke access because one data source is temporarily unavailable.

24. SECURITY AND PRIVACY

Follow current OWASP application/API guidance and applicable privacy requirements.

Protect against:

- Broken object-level/function-level authorization
- Cross-tenant and cross-property data exposure
- Authentication abuse and OTP brute force
- Session theft/fixation and privilege escalation
- CSRF where applicable
- XSS and SQL injection
- SSRF
- Mass assignment
- Unrestricted resource consumption
- Payment spoofing and webhook replay
- Duplicate event processing
- Voucher guessing, sharing abuse, and redemption races
- Router credential exposure
- Insecure direct network access
- Unsafe file uploads if uploads are introduced

Use strict validation, output encoding, security headers, safe CORS rules, rate limiting, dependency scanning, secret scanning, and least privilege. Never let a request select an arbitrary gateway URL, internal IP, filesystem path, or shell command. Never accept router commands directly from the browser.

Encrypt traffic in transit and protect secrets at rest. Avoid logging OTPs, voucher codes, session cookies, payment secrets, or router credentials. Minimize collection of device identifiers and network metadata; document purpose and retention. Provide privacy notice and a process for handling user data requests. Keep audit logs for privileged actions and document incident response and credential rotation.

Perform a threat model for:

- Voucher theft/guessing
- A group purchaser sharing a code outside the allowed group
- Payment spoofing or duplicate webhook
- Malicious tenant attempting cross-property access
- Compromised staff account
- Compromised router credentials
- Cloud outage while the local gateway remains active
- Router outage while the cloud application remains healthy
- Abuse of captive portal redirects

- Fraudulent refund or administrator-issued voucher

25. OBSERVABILITY AND RECOVERY

Provide:

- Liveness and readiness checks
- Database health
- Queue health and backlog
- Payment provider health where testable
- Gateway reachability and last successful check
- Worker heartbeat
- Provisioning failure rate
- Payment-to-activation latency
- Expiry enforcement failures
- Webhook processing failures
- Reconciliation discrepancies
- Error tracking and alert routing

Use structured JSON logs with correlation IDs linking order, payment, voucher, subscription, entitlement, and network operation. Redact secrets and personal data. Do not expose internal stack traces to customers.

Document recovery from:

- Application restart
- Worker crash
- Redis interruption
- Database interruption
- Router disconnection/reboot
- Starlink outage
- Payment provider outage
- Duplicate/out-of-order webhooks
- Missed expiry job
- Invalid gateway credentials
- Partial provisioning
- Failed refund
- Database restoration
- Lost/rotated secrets

Preserve financial truth during outages. Network availability and payment availability are separate. Define the local gateway’s behavior when the cloud control plane is unreachable: existing authorized sessions may continue only according to the configured gateway policy; new activations may need to fail closed or use a documented limited offline policy. Do not invent this behavior; test it.

Provide an operator runbook for safely restoring access and a break-glass method that does not require exposing the gateway to the public internet.

26. TESTING STRATEGY

Use automated tests plus explicit real-provider/hardware tests.

Unit tests

- Plan eligibility and price calculation
- Duration and expiry calculations, including boundaries and calendar months
- Voucher generation/verifier and state transitions
- Concurrent redemption/device limits
- Scope and group validation
- Subscription transitions
- Permissions
- Refund policy
- Network policy mapping

Integration tests

- Database constraints and migrations
- Payment event processing
- Duplicate and out-of-order webhooks
- Concurrent payment callbacks
- Atomic entitlement/voucher creation
- Outbox/job scheduling
- Queue retries and failed jobs
- Gateway adapter behavior
- Expiry/revocation
- Reconciliation

API tests

- Authentication and authorization
- Input validation and rate limits
- Cross-tenant isolation
- Error responses and pagination
- Idempotency
- Unauthorized network operations
- Voucher brute-force controls

End-to-end tests

- Tenant/guest onboarding as configured
- Plan selection and order creation
- Payment initiation and verified success
- Voucher issuance
- Voucher redemption
- Entitlement creation

- Network provisioning and actual connection
- Renewal
- Expiry
- Suspension and revocation
- Refund flow
- Gateway outage and recovery
- Pending payment/provisioning recovery

Security tests

- Unauthorized administrator access
- IDOR/BOLA attempts
- OTP brute force
- Forged/replayed/duplicate webhook
- Amount/currency/recipient mismatch
- Concurrent voucher redemption
- Unauthorized gateway selection
- Cross-property access
- Secret leakage
- Rate-limit bypass attempts

UI tests

- Mobile breakpoints
- Captive portal states
- Form validation and recovery
- Keyboard/screen-reader accessibility
- Loading, empty, error, and success states
- Responsive tables
- Payment-pending and provisioning-pending recovery

Use fake payment providers and fake network adapters for isolated automated tests. Also run sandbox tests against the real payment provider and separate physical gateway tests. Clearly label test doubles. Do not count mocked integration as proof of real-world operation.

27. PERFORMANCE AND CAPACITY

First document the expected number of tenants, concurrent devices, buildings, gateways, and transactions. Establish measurable targets based on that information rather than inventing them.

- Index query paths and inspect query plans.
- Avoid N+1 queries.
- Paginate large collections.
- Cache public plan data where appropriate, but never let stale caches override payment, entitlement, voucher, or authorization truth.
- Keep network operations out of ordinary synchronous page requests.
- Apply timeouts and bounded concurrency to external providers.

- Test database connection limits and worker concurrency.
- Load-test purchase, webhook, voucher redemption, login, expiry, and provisioning flows.
- Measure frontend Core Web Vitals on representative mobile devices.
- Measure gateway capacity, Wi-Fi coverage, and throughput at representative peak load.
- Do not report performance targets as achieved without measurements.

28. DEPLOYMENT AND INFRASTRUCTURE

Provide development, test, staging, and production environments. They must have separated credentials and data.

Deliver:

- `.env.example` with placeholders only
- Runtime configuration validation that fails fast for missing production variables
- Database migration instructions
- Safe seed script for demo data
- Local service configuration
- CI pipeline
- Deployment instructions
- Health-check configuration
- Backup schedule
- Restore procedure and restore-test evidence
- Rollback plan
- Secret rotation guide
- Production launch checklist
- Network installation/configuration guide
- Operator handover documentation

Use managed PostgreSQL with automated backups and point-in-time recovery where available. Use managed Redis or a secured production deployment. Deploy API and worker processes with independent restart/scaling policies. Use TLS on public endpoints. Restrict management interfaces and internal service access. Use a dedicated restricted router-management path.

Never run destructive database resets in production. Document migration compatibility and safe deployment order. A rollback must consider schema compatibility, queued jobs, payment events, and gateway state; reverting application code alone may not reverse a network change.

29. REQUIRED USER INTERFACES

Build three coordinated experiences.

A. Public marketing website

A polished, clear landing experience that explains the service, how vouchers work, plans, coverage limitations, support, terms, and privacy. Use real approved copy and avoid fake testimonials or invented statistics.

B. Captive portal and tenant experience

Extremely fast, mobile-first, resilient on poor connections, accessible, and usable on small screens. The most important actions are enter voucher, select a plan, pay, check status, and recover from a failed/pending activation.

C. Admin and operations

A restrained, efficient interface for plans, vouchers, tenants, room/floor groups, payments, subscriptions, gateway status, provisioning, reconciliation, reports, support, permissions, and audit. Make dangerous actions difficult to trigger accidentally.

All three must use the same design tokens, typography, iconography, and interaction principles.

30. IMPLEMENTATION PHASES AND EXIT CRITERIA

Phase 0 — Discovery and feasibility

Repository audit, hardware discovery, official documentation review, assumptions/risk register, legal/plan gate checklist, architecture proposal, capability matrix. **Exit:** owner decisions and blockers documented; no invented hardware assumptions.

Phase 1 — Foundation

Workspace/build setup, strict TypeScript, lint/typecheck/test/build scripts, config validation, database, migrations, CI, design tokens, logging/correlation IDs. **Exit:** reproducible local setup and green baseline checks.

Phase 2 — Identity and property model

Authentication as approved, roles/permissions, property/building/floor/room, tenant assignments, group membership, audit. **Exit:** permission and cross-property tests pass.

Phase 3 — Plans, vouchers, and orders

Plan versioning, scope rules, duration policy, voucher model, issuance/redeem state machine, server pricing, orders, idempotency. **Exit:** voucher concurrency and eligibility tests pass.

Phase 4 — Payment engine

Provider abstraction, Paystack sandbox integration, webhook verification, transaction verification, event persistence, refunds, reconciliation, outbox. **Exit:** duplicate, forged, mismatched, and out-of-order event tests pass; real sandbox flow verified.

Phase 5 — Entitlement engine

Subscription state transitions, group policy, device limits, activation/expiry, renewal, suspension, cancellation, revocation. **Exit:** expiry boundaries, group access, and recovery tests pass.

Phase 6 — Hardware proof-of-concept and network integration

Select gateway, validate hardware/firmware, prove captive portal and voucher enforcement, implement real adapter, queue, policy mapping, session inspection, revocation, reconciliation. **Exit:** physical gateway tests prove actual connection and expiry/revocation. If hardware is unavailable, mark this phase blocked/live-unverified rather than complete.

Phase 7 — Tenant experience

Marketing, captive portal, tenant portal, receipts, device/session management, support and recovery. **Exit:** UI tests pass and representative mobile/captive-portal behavior is tested.

Phase 8 — Admin experience

Operations dashboard, plan/voucher management, payment reconciliation, network controls, reports, permissions, audit logs. **Exit:** all mutations are authorized, audited, validated, and tested.

Phase 9 — Hardening

Security review, concurrency tests, load tests, backup restore, failure/recovery exercises, accessibility and mobile testing, operational alerts. **Exit:** no unresolved critical security defects; agreed targets measured.

Phase 10 — Staging and launch

Sandbox payments, real hardware acceptance tests, owner acceptance testing, production deployment, monitoring, documentation, and handover. **Exit:** launch checklist signed off and capability matrix completed.

Do not start the next major phase while the previous phase has unresolved critical defects. Independent work may proceed only where it does not depend on unresolved decisions.

31. DEFINITION OF DONE

The product is not complete merely because pages look finished or the app builds.

Completion requires:

- Successful production build
- Passing lint and type checks
- Applied, reproducible database migrations
- Passing unit, integration, API, and end-to-end tests
- Verified server-side authorization and cross-property isolation
- Verified payment-provider sandbox integration
- Valid webhook signature handling and duplicate-delivery safety
- Verified payment-to-order/voucher/entitlement consistency
- Voucher issuance, redemption, expiry, and revocation tests
- Actual gateway provisioning and revocation on compatible physical hardware
- Actual expiry enforcement and real client internet access
- Verified room/floor scope and device limits
- Verified mobile/captive-portal behavior
- No unresolved critical security findings
- Working logs, alerts, health checks, and reconciliation
- Verified backup restoration
- Documented recovery procedures
- Deployment instructions and environment documentation
- Owner-approved legal/Starlink/network readiness checks
- Complete known-limitations and risk register
- A capability matrix distinguishing live-verified, implemented but not live-verified, simulated-only, blocked, and not implemented
- Clear list of remaining tasks and launch blockers

If a physical gateway, payment credential, SMS credential, legal approval, or production domain is unavailable, build what can safely be built and isolate simulations. Do not label the overall system fully production-validated.

32. REQUIRED FINAL HANDOVER

Deliver and maintain:

1. Architecture overview and data-flow diagrams.
2. Repository structure and technology rationale.
3. Discovery report and hardware capability matrix.
4. Assumptions register and risk register.
5. Database schema, constraints, and migration history.
6. API specification/OpenAPI document.
7. Voucher and entitlement state-machine documentation.
8. Network installation and gateway configuration guide.
9. Physical gateway proof-of-concept test report.
10. Payment setup, webhook, reconciliation, and refund guide.
11. Environment variable reference with no secrets.
12. Automated test results with actual commands and outcomes.
13. Security checklist and threat model.
14. Deployment, rollback, backup, and recovery runbooks.
15. Administrator guide.
16. Tenant onboarding and voucher-recovery guide.
17. Monitoring/alert guide.
18. Known limitations and risk register.
19. Verified implementation/capability matrix.
20. Outstanding tasks, launch blockers, and owner actions.

33. FIRST RESPONSE REQUIRED FROM THE CODING AGENT

Before writing substantial code, return:

1. A concise repository audit based on files actually inspected.
2. A list of known decisions and unresolved assumptions.
3. The proposed architecture and why it fits the repository.
4. The hardware/provider feasibility checklist.
5. A capability matrix with unknowns clearly marked.
6. The highest-risk security, payment, legal, and network failure modes.
7. A phased plan with phase exit criteria.
8. The first small implementation increment.
9. Commands/tests that will be run and the expected evidence.
10. Questions that require owner input, grouped by urgency.

Then begin Phase 0 deliverables. Do not start by generating a large amount of unverified UI. Build the foundation incrementally, keep tests passing, and maintain truthful progress reports.

Final instruction: Build NetFlex as a real, maintainable network-access business platform, not a visual demo. Preserve financial correctness, enforce access at the actual gateway, make voucher and expiry behavior explicit, design for mobile-first use, and never claim functionality that has not been verified.